

Experiment No. 1

Importing Libraries

```
In [1]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

Loading the Dataset

First we load the dataset and find out the number of columns, rows, NULL values, etc.

1. Data is from 2009 to 2015
2. 200,000 Entries

```
In [2]: uber = pd.read_csv('uber.csv')
uber.head()
```

Out[2]:

	Unnamed: 0	key	fare_amount	pickup_datetime	pickup_longitude	pick
0	24238194	2015-05-07 19:52:06.0000003	7.5	2015-05-07 19:52:06 UTC	-73.999817	
1	27835199	2009-07-17 20:04:56.0000002	7.7	2009-07-17 20:04:56 UTC	-73.994355	
2	44984355	2009-08-24 21:45:00.00000061	12.9	2009-08-24 21:45:00 UTC	-74.005043	
3	25894730	2009-06-26 08:22:21.0000001	5.3	2009-06-26 08:22:21 UTC	-73.976124	
4	17610152	2014-08-28 17:47:00.000000188	16.0	2014-08-28 17:47:00 UTC	-73.925023	



```
In [3]: uber.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200000 entries, 0 to 199999
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0            200000 non-null  int64
1   key                   200000 non-null  object
2   fare_amount           200000 non-null  float64
3   pickup_datetime       200000 non-null  object
4   pickup_longitude      200000 non-null  float64
5   pickup_latitude       200000 non-null  float64
6   dropoff_longitude     199999 non-null  float64
7   dropoff_latitude      199999 non-null  float64
8   passenger_count       200000 non-null  int64
dtypes: float64(5), int64(2), object(2)
memory usage: 13.7+ MB

```

Dropping the NULL values

We will check for NULL values in Dataset

```
In [4]: uber.isnull().sum()
```

```

Out[4]: Unnamed: 0      0
        key           0
        fare_amount    0
        pickup_datetime 0
        pickup_longitude 0
        pickup_latitude 0
        dropoff_longitude 1
        dropoff_latitude 1
        passenger_count 0
        dtype: int64

```

Getting rid of first and second column, since key and ID are not useful in predictions.

```
In [5]: uber_2 = uber.drop(['Unnamed: 0', 'key'], axis=1)
        uber_2.dropna(axis=0, inplace=True)
```

We have gotten rid of the first two columns and the NULL values

```
In [6]: uber_2.isnull().sum()
        #uber_2.describe()
```

```

Out[6]: fare_amount      0
        pickup_datetime  0
        pickup_longitude  0
        pickup_latitude  0
        dropoff_longitude 0
        dropoff_latitude 0
        passenger_count   0
        dtype: int64

```

Haversine Formula

Calculatin the distance between the pickup and drop co-ordinates using the Haversine formual for accuracy.

~~

```
In [7]: def haversine (lon_1, lon_2, lat_1, lat_2):

        lon_1, lon_2, lat_1, lat_2 = map(np.radians, [lon_1, lon_2, lat_1, lat_2])

        diff_lon = lon_2 - lon_1
        diff_lat = lat_2 - lat_1

        km = 2 * 6371 * np.arcsin(np.sqrt(np.sin(diff_lat/2.0)**2 +
                                           np.cos(lat_1) * np.cos(lat_2) * np.sin(diff
                                           lon/2.0)**2))

        return km
```

Defining the ride distance dataframe.

```
In [8]: uber_2['Distance'] = haversine(uber_2['pickup_longitude'],uber_2['dropoff_longitude'],
                                       uber_2['pickup_latitude'],uber_2['dropoff_latitude'])

uber_2['Distance'] = uber_2['Distance'].astype(float).round(2)
```

```
In [9]: uber_2.head()
```

```
Out[9]:
```

	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude
0	7.5	2015-05-07 19:52:06 UTC	-73.999817	40.738354	-73.999512
1	7.7	2009-07-17 20:04:56 UTC	-73.994355	40.728225	-73.994710
2	12.9	2009-08-24 21:45:00 UTC	-74.005043	40.740770	-73.962565
3	5.3	2009-06-26 08:22:21 UTC	-73.976124	40.790844	-73.965316
4	16.0	2014-08-28 17:47:00 UTC	-73.925023	40.744085	-73.973082

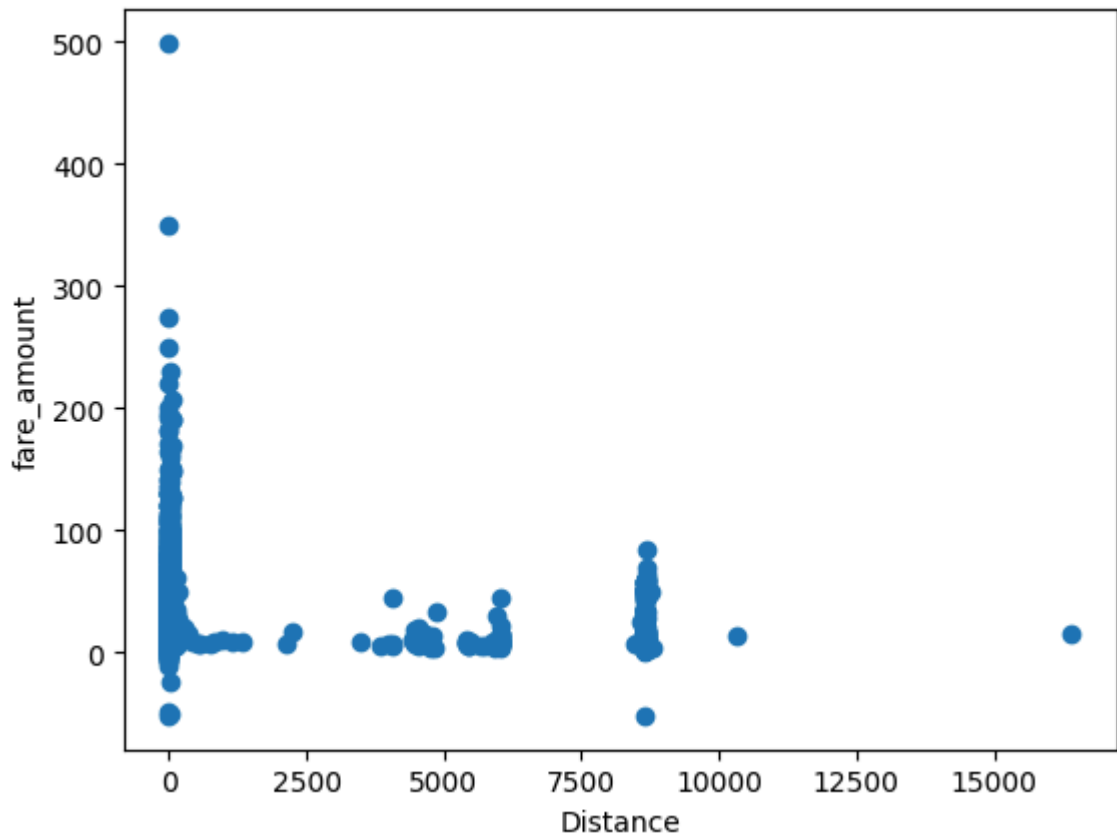


Scatter Plot

Distance vs Fare Amount

```
In [10]: plt.scatter(uber_2['Distance'], uber_2['fare_amount'])
plt.xlabel("Distance")
plt.ylabel("fare_amount")
```

```
Out[10]: Text(0, 0.5, 'fare_amount')
```



Outliers

We can get rid of the trips with very large distances that are outliers as well as trips with 0 distance.

```
In [11]: uber_2.drop(uber_2[uber_2['Distance'] > 60].index, inplace = True)
uber_2.drop(uber_2[uber_2['Distance'] == 0].index, inplace = True)
uber_2.drop(uber_2[uber_2['fare_amount'] == 0].index, inplace = True)
uber_2.drop(uber_2[uber_2['fare_amount'] < 0].index, inplace = True)
```

Also removing rows with non-plausible fare amounts and distance travelled

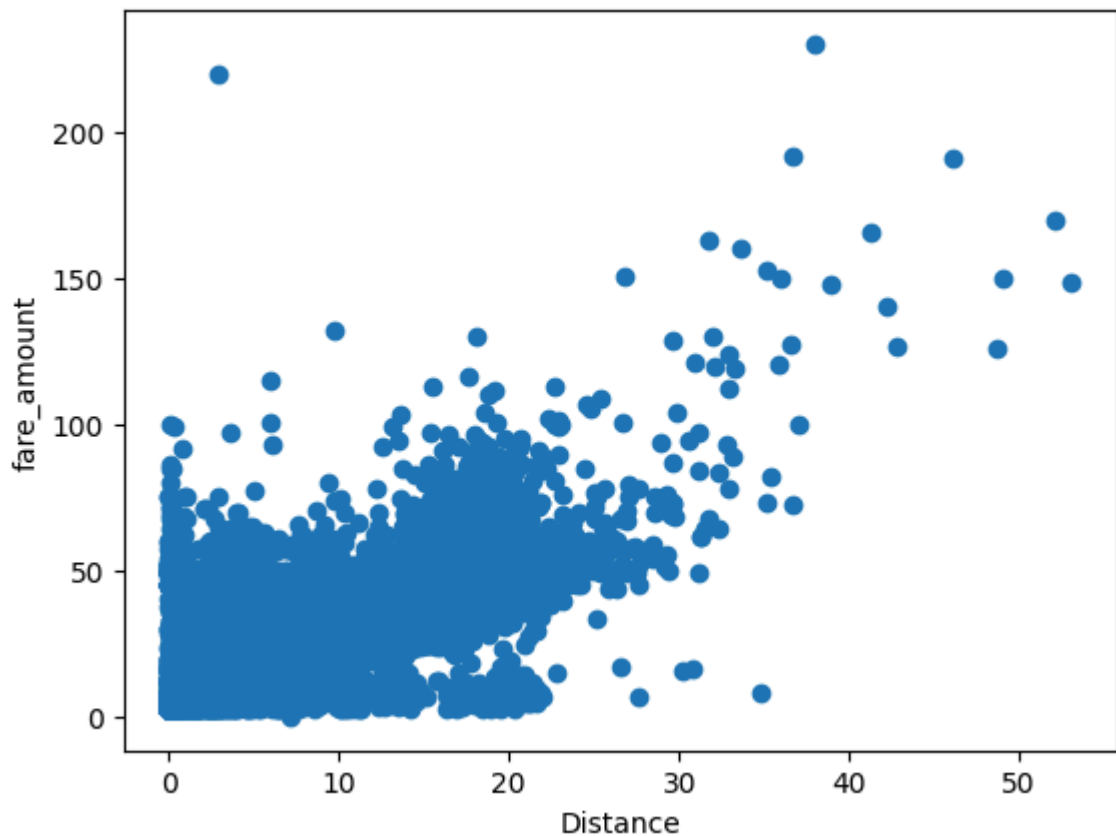
```
In [12]: uber_2.drop(uber_2[(uber_2['fare_amount'] > 100) & (uber_2['Distance'] < 1)].index,
uber_2.drop(uber_2[(uber_2['fare_amount'] < 100) & (uber_2['Distance'] > 100)].index)
```

```
In [13]: uber_2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 193481 entries, 0 to 199999
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   fare_amount            193481 non-null float64
1   pickup_datetime        193481 non-null object
2   pickup_longitude       193481 non-null float64
3   pickup_latitude        193481 non-null float64
4   dropoff_longitude      193481 non-null float64
5   dropoff_latitude       193481 non-null float64
6   passenger_count        193481 non-null int64
7   Distance               193481 non-null float64
dtypes: float64(6), int64(1), object(1)
memory usage: 13.3+ MB
```

```
In [14]: plt.scatter(uber_2['Distance'], uber_2['fare_amount'])
plt.xlabel("Distance")
plt.ylabel("fare_amount")
```

```
Out[14]: Text(0, 0.5, 'fare_amount')
```



Now the scatter plot is looking more suitable.

Date and Time

Separating the date and time into separate columns for more usability.

```
In [15]: uber_2['pickup_datetime'] = pd.to_datetime(uber_2['pickup_datetime'])

uber_2['Year'] = uber_2['pickup_datetime'].apply(lambda time: time.year)
uber_2['Month'] = uber_2['pickup_datetime'].apply(lambda time: time.month)
uber_2['Day'] = uber_2['pickup_datetime'].apply(lambda time: time.day)
```

```

uber_2['Day of Week'] = uber_2['pickup_datetime'].apply(lambda time: time.dayofw
uber_2['Day of Week_num'] = uber_2['pickup_datetime'].apply(lambda time: time.da
uber_2['Hour'] = uber_2['pickup_datetime'].apply(lambda time: time.hour)

day_map = {0:'Mon',1:'Tue',2:'Wed',3:'Thu',4:'Fri',5:'Sat',6:'Sun'}
uber_2['Day of Week'] = uber_2['Day of Week'].map(day_map)

uber_2['counter'] = 1

```

Pickup and Dropoff Columns

Creating separate coumns for pickup and droppoff coordinates for more usability.

```

In [16]: uber_2['pickup'] = uber_2['pickup_latitude'].astype(str) + "," + uber_2['pickup_
uber_2['drop off'] = uber_2['dropoff_latitude'].astype(str) + "," + uber_2['drop

```

```

In [17]: uber_2.head()

```

Out[17]:

	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude
0	7.5	2015-05-07 19:52:06+00:00	-73.999817	40.738354	-73.999512
1	7.7	2009-07-17 20:04:56+00:00	-73.994355	40.728225	-73.994710
2	12.9	2009-08-24 21:45:00+00:00	-74.005043	40.740770	-73.962565
3	5.3	2009-06-26 08:22:21+00:00	-73.976124	40.790844	-73.965316
4	16.0	2014-08-28 17:47:00+00:00	-73.925023	40.744085	-73.973082

Thus, we have increased the usability of the dataset.

Data Visualizations

Finding the trends in the data variables

Average Yearly Trips

```

In [18]: no_of_trips = []
year = [2009, 2010, 2011, 2012, 2013, 2014, 2015]

colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd',
          '#8c564b', '#e377c2', '#7f7f7f', '#bcbd22', '#17becf']

for i in range(2009, 2016):
    x = uber_2.loc[uber_2['Year'] == i, 'counter'].sum()
    no_of_trips.append(x)

print("Average trips a year: ")

```

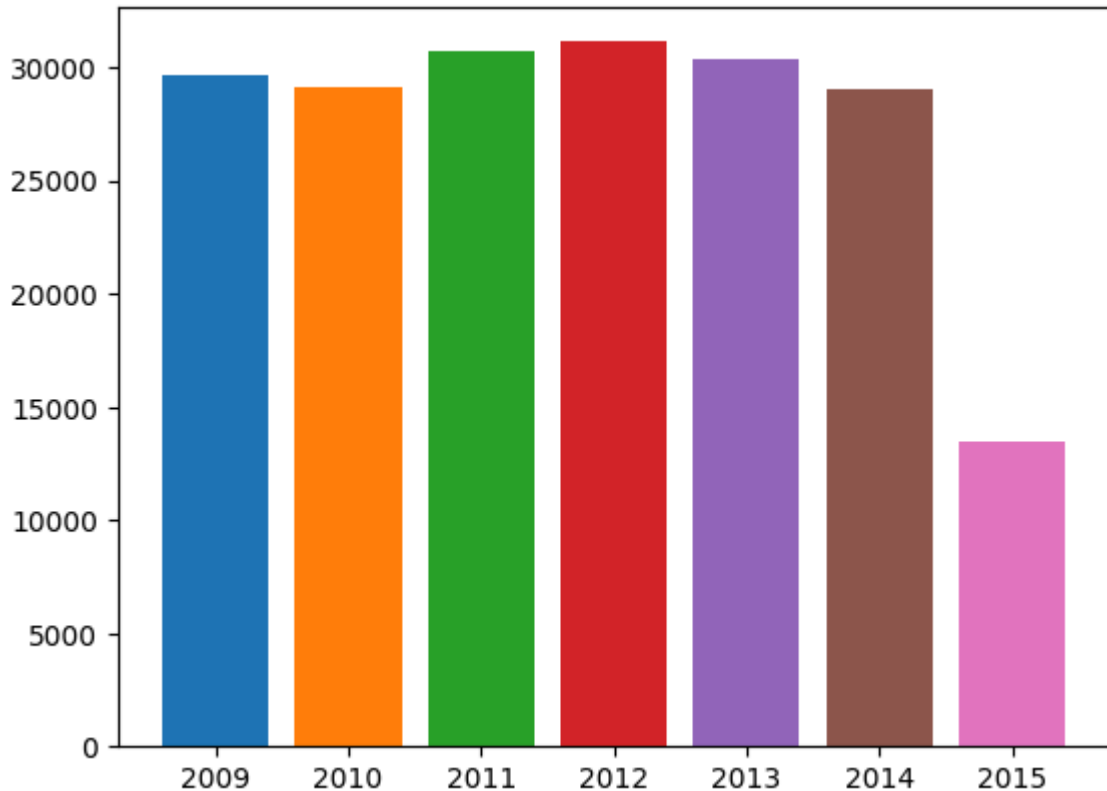
```
print(year, no_of_trips)

plt.bar(year, no_of_trips, color=colors)
```

Average trips a year:

```
[2009, 2010, 2011, 2012, 2013, 2014, 2015] [np.int64(29674), np.int64(29094), np.int64(30712), np.int64(31138), np.int64(30365), np.int64(29065), np.int64(13433)]
```

Out[18]: <BarContainer object of 7 artists>



Average Monthly Trips

```
In [19]: no_of_trips = []
month = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd',
          '#8c564b', '#e377c2', '#7f7f7f', '#bcbd22', '#17becf']

for i in range(1, 13):
    x = uber_2.loc[uber_2['Month'] == i, 'counter'].sum()
    no_of_trips.append(x)

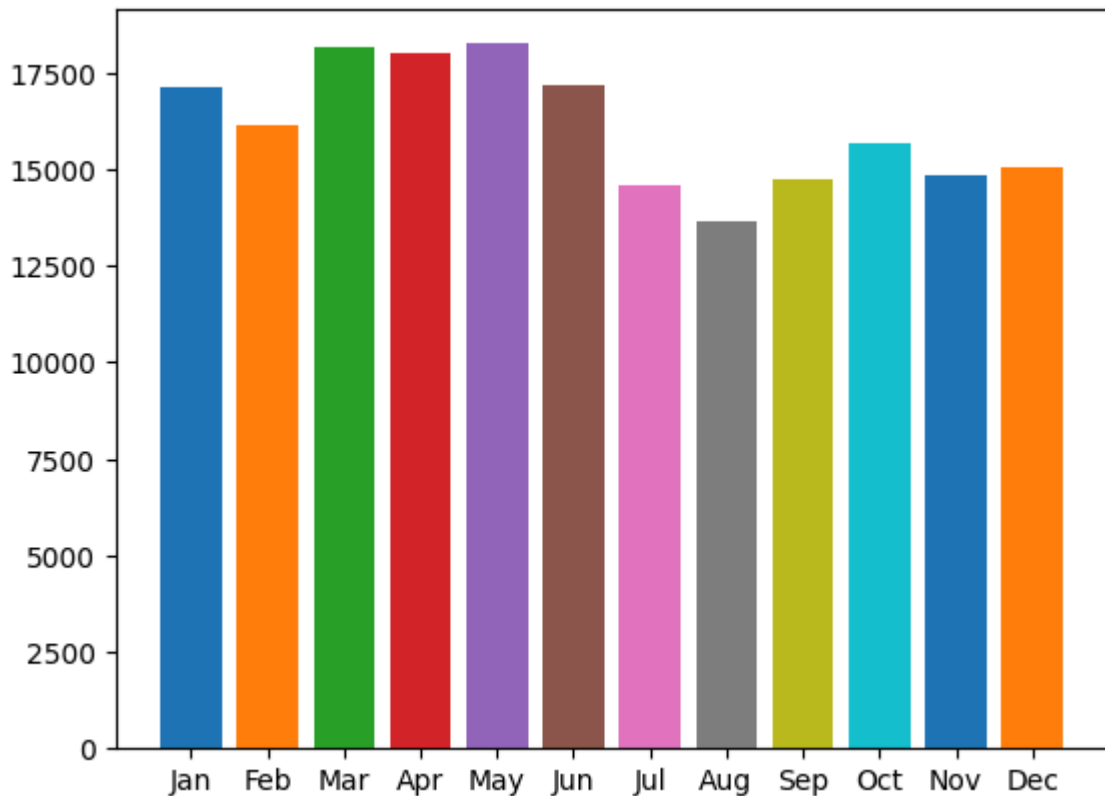
print("Average trips a Month: ")
print(month, no_of_trips)

plt.bar(month, no_of_trips, color=colors)
```

Average trips a Month:

```
['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'] [np.int64(17126), np.int64(16141), np.int64(18165), np.int64(18004), np.int64(18259), np.int64(17210), np.int64(14583), np.int64(13664), np.int64(14772), np.int64(15690), np.int64(14824), np.int64(15043)]
```

Out[19]: <BarContainer object of 12 artists>



Average Daily Trips

```
In [20]: no_of_trips = []
day = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']

colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd',
          '#8c564b', '#e377c2', '#7f7f7f', '#bcbd22', '#17becf']

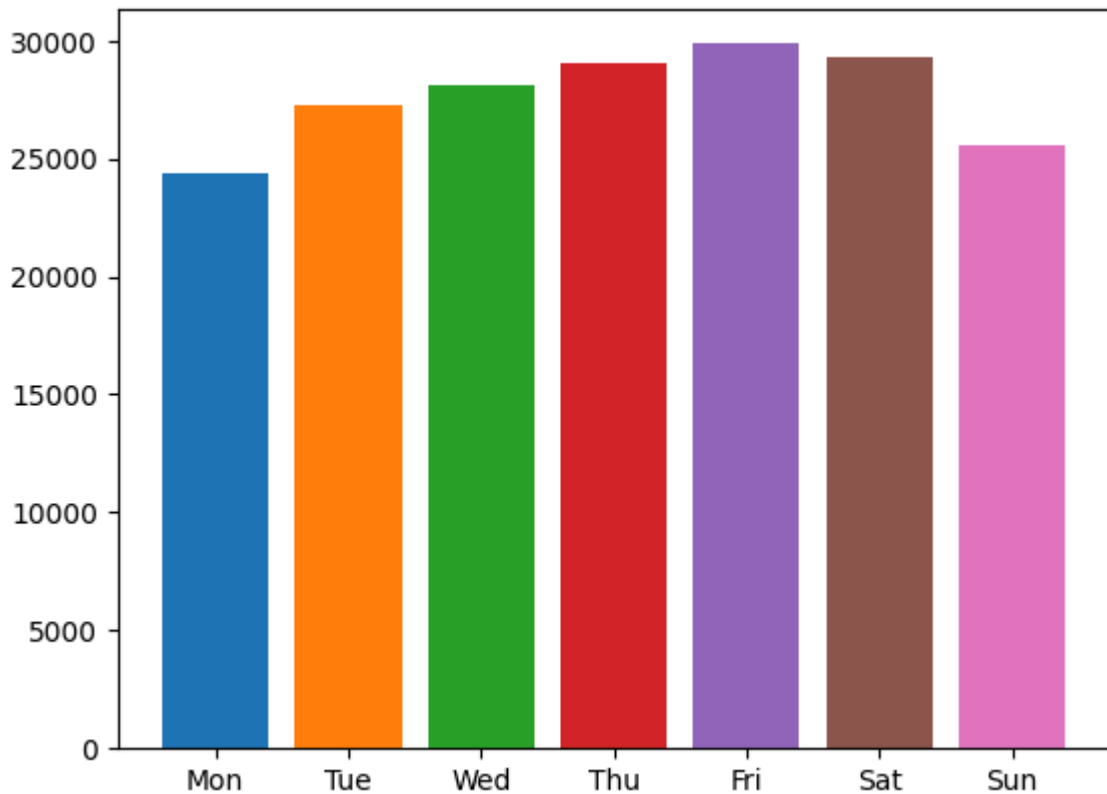
for i in range(0, 7):
    x = uber_2.loc[uber_2['Day of Week_num'] == i, 'counter'].sum()
    no_of_trips.append(x)

print("Average trips by Days: ")
print(day, no_of_trips)

plt.bar(day, no_of_trips, color=colors)
```

```
Average trips by Days:
['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun'] [np.int64(24377), np.int64(27236), np.int64(28084), np.int64(29041), np.int64(29865), np.int64(29306), np.int64(25572)]
```

```
Out[20]: <BarContainer object of 7 artists>
```



Rides vs Time

Relation between average number of rides over a period of time.

```
In [21]: year_vs_trips = uber_2.groupby(['Year', 'Month']).agg(
    no_of_trips = ('counter', 'count'),
    Average_fair = ('fare_amount', 'mean'),
    Total_fair = ('fare_amount', 'sum'),
    Avg_distance = ('Distance', 'mean')).reset_index()

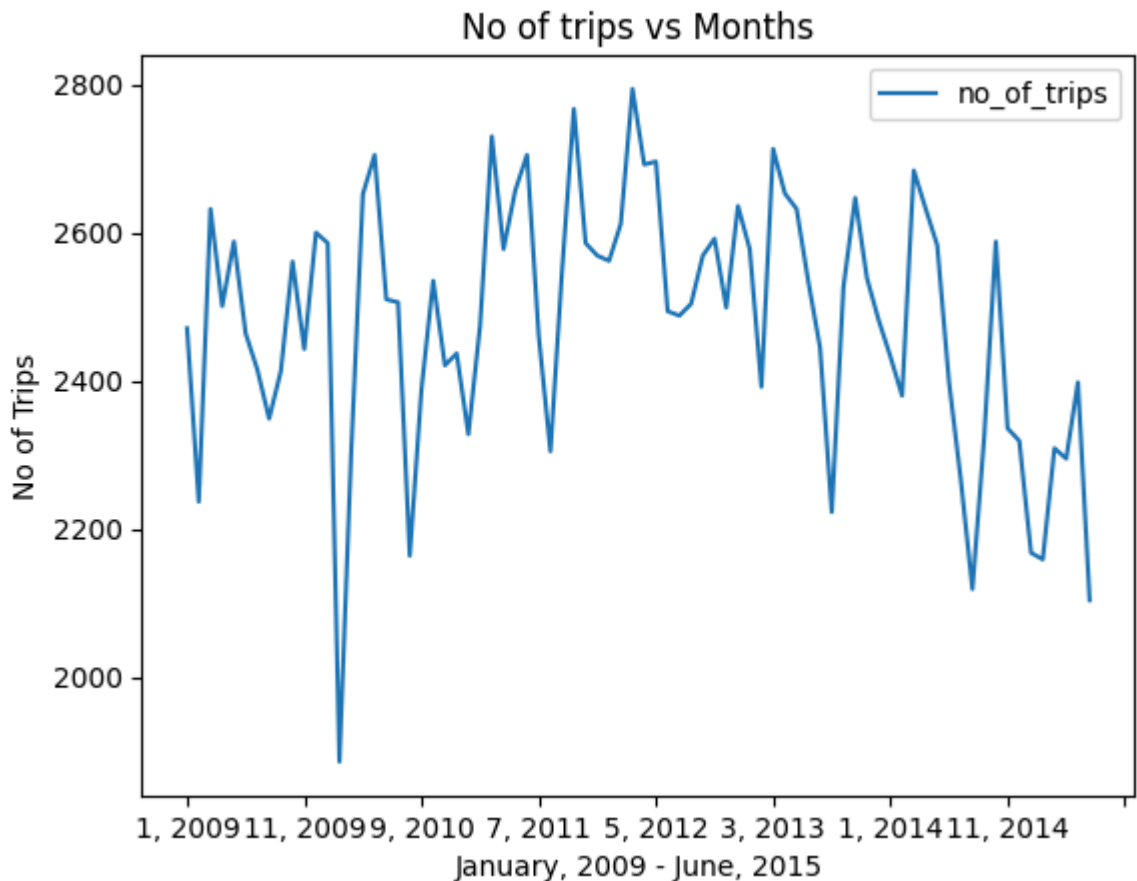
year_vs_trips['avg_no_of_trips'] = year_vs_trips['no_of_trips']/30
year_vs_trips['month_year'] = year_vs_trips['Month'].astype(str) + ", " + year_vs_

year_vs_trips = year_vs_trips.reset_index()

year_vs_trips.head()

year_vs_trips.plot(kind='line', x='month_year', y='no_of_trips', xlabel='January,
    ylabel='No of Trips', title='No of trips vs Months')
```

```
Out[21]: <Axes: title={'center': 'No of trips vs Months'}, xlabel='January, 2009 - June,
    2015', ylabel='No of Trips'>
```



Heat-Map

A heat map to illustrate at what time of day and week, people are using Uber the most.

In [22]: `import seaborn as sns`

```
df_1 = uber_2[['Distance', 'Day of Week_num', 'Hour']].copy()
```

```
df_h = df_1.copy()
```

```
df_h = df_h.groupby(['Hour', 'Day of Week_num']).mean()
```

```
df_h = df_h.unstack(level=0)
```

In [23]: `fig, ax = plt.subplots(figsize=(24, 7))`

```
sns.heatmap(df_h, cmap="Reds",
            linewidth=0.3, cbar_kws={"shrink": .8})
```

```
xticks_labels = ['12 AM', '01 AM', '02 AM ', '03 AM ', '04 AM ', '05 AM ', '06 A',
                  '08 AM ', '09 AM ', '10 AM ', '11 AM ', '12 PM ', '01 PM ', '02',
                  '04 PM ', '05 PM ', '06 PM ', '07 PM ', '08 PM ', '09 PM ', '10
```

```
yticks_labels = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
```

```
plt.xticks(np.arange(24) + .5, labels=xticks_labels)
```

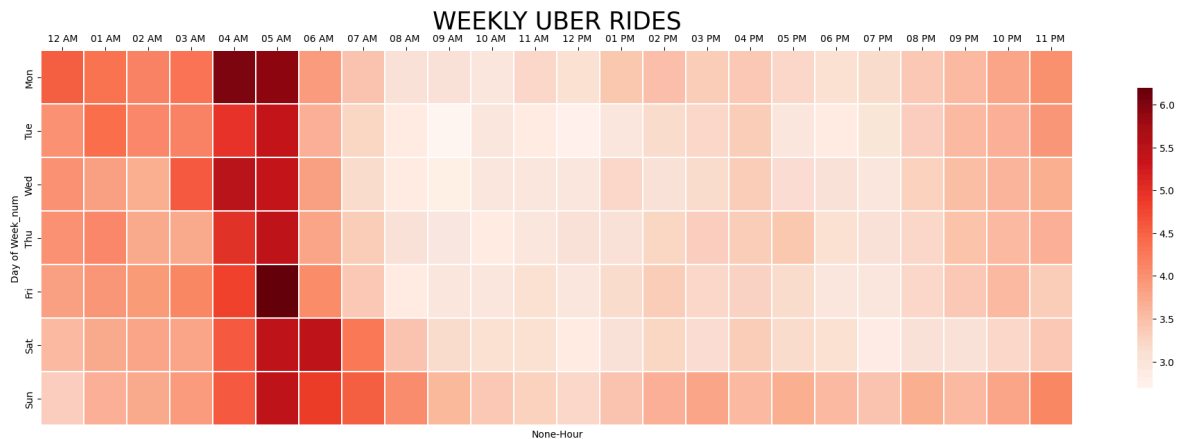
```
plt.yticks(np.arange(7) + .5, labels=yticks_labels)
```

```
ax.xaxis.tick_top()
```

```
title = 'Weekly Uber Rides'.upper()
```

```
plt.title(title, fontdict={'fontsize': 25})
```

```
plt.show()
```



```
In [24]: plt.figure(figsize=[8,4])
sns.distplot(uber_2['fare_amount'], color='g', hist_kws=dict(edgecolor="black", linewidth=2), bins=30)
plt.title('Target Variable Distribution')
plt.show()
```

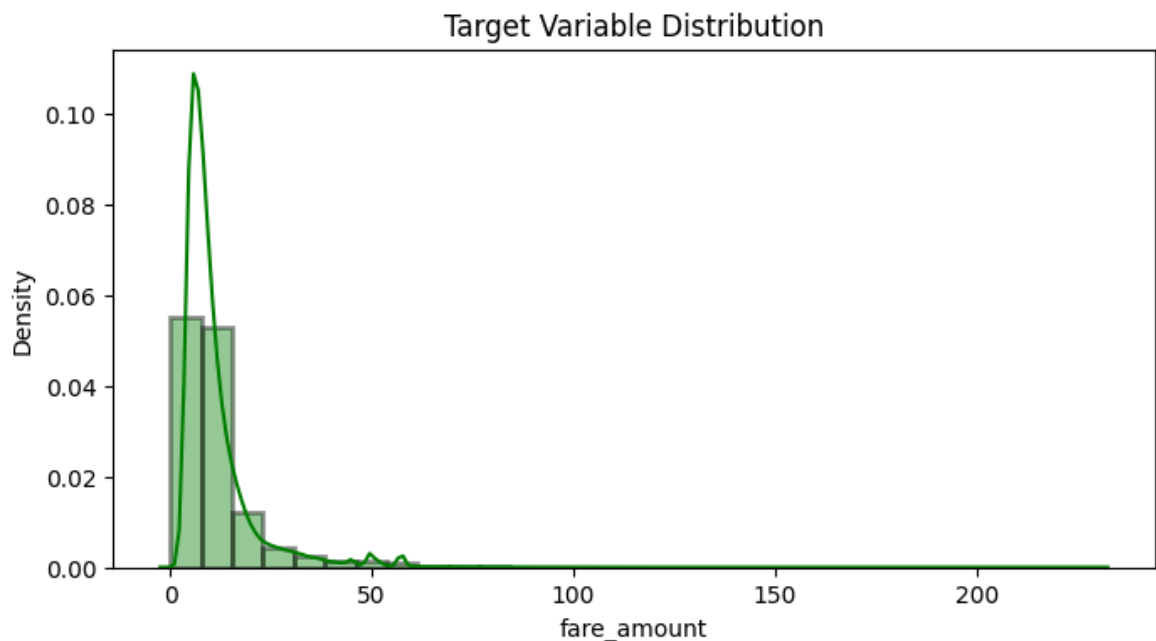
C:\Users\swaro\AppData\Local\Temp\ipykernel_19460\838148572.py:2: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(uber_2['fare_amount'], color='g', hist_kws=dict(edgecolor="black", linewidth=2), bins=30)
```



Statistics

Some general statistical information about the data

Fare Amount

```
In [25]: import statistics as st

print("Mean of fare prices is % s "
      % (st.mean(uber_2['fare_amount'])))

print("Median of fare prices is % s "
      % (st.median(uber_2['fare_amount'])))

print("Standard Deviation of Fare Prices is % s "
      % (st.stdev(uber_2['fare_amount'])))
```

Mean of fare prices is 11.311137476031238
Median of fare prices is 8.5
Standard Deviation of Fare Prices is 9.501354886915946

Distance

```
In [26]: import statistics as st

print("Mean of Distance is % s "
      % (st.mean(uber_2['Distance'])))

print("Median of Distance is % s "
      % (st.median(uber_2['Distance'])))

print("Standard Deviation of Distance is % s "
      % (st.stdev(uber_2['Distance'])))
```

Mean of Distance is 3.3573488352861522
Median of Distance is 2.18
Standard Deviation of Distance is 3.5995030824386185

Corelation Matrix

To find the two variables that have the most inter-dependence

```
In [28]: corr = uber_2.corr(numeric_only=True)
corr.style.background_gradient(cmap="BuGn")
```

```
d:\ml_env\Lib\site-packages\pandas\io\formats\style.py:3807: RuntimeWarning: All-
NaN slice encountered
  smin = np.nanmin(gmap) if vmin is None else vmin
d:\ml_env\Lib\site-packages\pandas\io\formats\style.py:3808: RuntimeWarning: All-
NaN slice encountered
  smax = np.nanmax(gmap) if vmax is None else vmax
```

Out[28]:

	fare_amount	pickup_longitude	pickup_latitude	dropoff_longitude	c
fare_amount	1.000000	0.012070	-0.008682	0.010555	
pickup_longitude	0.012070	1.000000	-0.949099	0.999883	
pickup_latitude	-0.008682	-0.949099	1.000000	-0.949095	
dropoff_longitude	0.010555	0.999883	-0.949095	1.000000	
dropoff_latitude	-0.008695	-0.993975	0.954759	-0.993962	
passenger_count	0.013508	0.009174	-0.009219	0.009168	
Distance	0.893050	0.005337	0.003232	0.004406	
Year	0.123562	0.013480	-0.013690	0.013365	
Month	0.024402	-0.007496	0.007602	-0.007454	
Day	-0.000286	0.019528	-0.019390	0.019554	
Day of Week_num	0.005140	0.008241	-0.008922	0.008536	
Hour	-0.019788	0.001838	-0.001821	0.000938	
counter	nan	nan	nan	nan	

There is some coorelation between the distance and fare amount. So we will implement our simple linear regression model using these two varaibles

Standardization

For more accurate results on our linear regression model

Assigning the dependent and independent variable

```
In [29]: X = uber_2['Distance'].values.reshape(-1, 1)      #Independent Variable
          y = uber_2['fare_amount'].values.reshape(-1, 1)  #Dependent Variable
```

```
In [30]: from sklearn.preprocessing import StandardScaler
std = StandardScaler()
y_std = std.fit_transform(y)
print(y_std)

x_std = std.fit_transform(X)
print(x_std)
```

```
[[-0.4011162 ]
 [-0.38006651]
 [ 0.16722527]
 ...
 [ 2.06169682]
 [ 0.33562274]
 [ 0.29352337]]
[[-0.46599576]
 [-0.24929862]
 [ 0.46746884]
 ...
 [ 2.63721838]
 [ 0.05074357]
 [ 0.57303924]]
```

Splitting the Dataset

Training and Test Set

```
In [31]: from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(x_std, y_std, test_size=0.2,
```

Simple Linear Regression

Training the simple linear regression model on the training set

```
In [32]: from sklearn.linear_model import LinearRegression
l_reg = LinearRegression()
l_reg.fit(X_train, y_train)

print("Training set score: {:.2f}".format(l_reg.score(X_train, y_train)))
print("Test set score: {:.7f}".format(l_reg.score(X_test, y_test)))
```

Training set score: 0.80

Test set score: 0.8006071

Actual vs Predicted Values

```
In [36]: import pandas as pd
from tabulate import tabulate

y_pred = l_reg.predict(X_test)

# Ensure both are 1D
y_test_flat = y_test.ravel() if hasattr(y_test, "ravel") else y_test
y_pred_flat = y_pred.ravel() if hasattr(y_pred, "ravel") else y_pred

df = pd.DataFrame({
    'Actual': y_test_flat,
    'Predicted': y_pred_flat
})

print(tabulate(df.head(10), headers='keys', tablefmt='psql'))
```

	Actual	Predicted
0	-0.548464	-0.663476
1	-0.769486	-0.611449
2	0.651368	0.290356
3	6.86102	3.52595
4	-0.611613	-0.544557
5	-0.16957	-0.333971
6	-0.295868	-0.0490599
7	-0.0853709	-0.224961
8	-0.506365	-0.517305
9	-0.558989	-0.628792

Accuracy Checking

Finding the MSE, MAE, RMSE, etc.

```
In [37]: from sklearn import metrics
print('Mean Absolute Error:', metrics.mean_absolute_error(y_test, y_pred))
#print('Mean Absolute % Error:', metrics.mean_absolute_percentage_error(y_test,
print('Mean Squared Error:', metrics.mean_squared_error(y_test, y_pred))
print('Root Mean Squared Error:', np.sqrt(metrics.mean_squared_error(y_test, y_p
```

Mean Absolute Error: 0.23979020298617207
Mean Squared Error: 0.19845894128014663
Root Mean Squared Error: 0.44548730765325584

Plotting the Graph

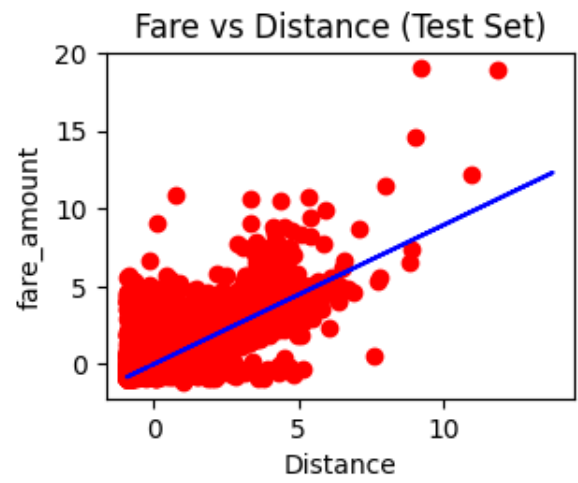
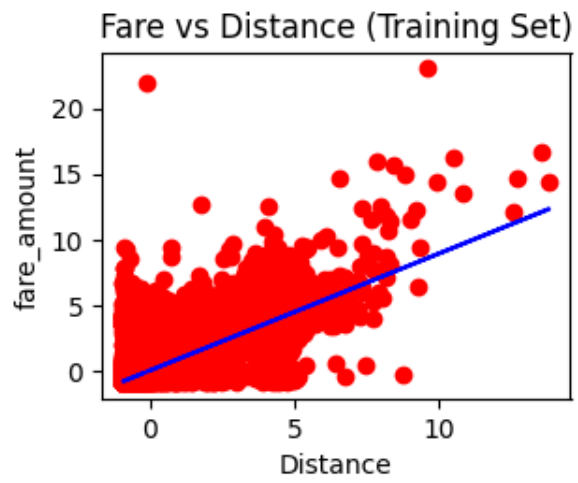
```
In [38]: print(l_reg.intercept_)
print(l_reg.coef_)
```

[-0.00016705]
[[0.89176936]]

```
In [39]: plt.subplot(2, 2, 1)
plt.scatter(X_train, y_train, color = 'red')
plt.plot(X_train, l_reg.predict(X_train), color = "blue")
plt.title("Fare vs Distance (Training Set)")
plt.ylabel("fare_amount")
plt.xlabel("Distance")

plt.subplot(2, 2, 2)
plt.scatter(X_test, y_test, color = 'red')
plt.plot(X_train, l_reg.predict(X_train), color = "blue")
plt.ylabel("fare_amount")
plt.xlabel("Distance")
plt.title("Fare vs Distance (Test Set)")

plt.tight_layout()
plt.rcParams["figure.figsize"] = (32,22)
plt.show()
```



In []: